

課題仕様書(Unity版): 固定シード式ターン制ローグライク

Claude Code を「矯正しながら使う」ことを習得するための課題。Unity 上で実装する場合の版。汎用版([roguelike-challenge.md](#))と学習目標・ゲーム仕様は共通だが、制約の機械判定手段が **Unity** 固有に置き換わる。本書は差分と Unity 固有の注意点に集中する。

各 Unity API 仕様は本書作成時点(2026-06)の公式ドキュメントで裏付けたが、エディタ/パッケージのバージョンにより細部が変わりうる。実際に組む前にプロジェクトの Unity バージョンに対応する公式ドキュメントを確認すること。

0. この文書の構成(地図)

1. なぜ Unity だと設計が変わるか — 原理
2. ゲーム仕様 — 汎用版との差分
3. アーキテクチャの必須要件 — コア層分離(Unity では任意ではなく必須)
4. 機械的制約 — Unity 固有の判定手段に差し替え
5. テスト基盤 — Unity Test Framework の Edit Mode
6. カバレッジ — Unity Code Coverage のバッチモード計測
7. 進め方・完成判定 — 汎用版からの差分
8. 想定される Claude の逃げと対処(Unity 版)

1. なぜ Unity だと設計が変わるか(原理)

汎用版の心臓は「決定論テスト(同一シードでリプレイヤー一致)」だった。Unity ではこの決定論が、フレームワークの標準的な書き方と正面衝突する。理由は二つ。

第一に、`Time.deltaTime` と `FixedUpdate` を使ったフレーム依存の進行は、フレームレートの揺れにより同一入力でも結果がぶれる。第二に、`Rigidbody` を含む物理エンジン(PhysX)は浮動小数の累積と内部処理順序により厳密な再現性を保証しない。

したがって **Unity** でこの課題を成立させるには、ゲームのコアロジックを `MonoBehaviour` と物理から完全に切り離れた純粋な **C#** 層に置くことが前提条件になる。汎用版で「任意(制約F)」としたコア層の I/O 分離が、Unity 版では必須要件に格上げされる。この分離自体が Unity で最も価値ある学習項目である。

2. ゲーム仕様(汎用版との差分)

ゲーム内容(固定シードのターン制ダンジョン探索、自動進行、勝敗判定、リプレイ記録)は汎用版と同一。Unity 固有の差分のみ記す。

- 進行をフレームに乗せない: ゲームの1ターンは `Update` のフレーム経過ではなく、コアロジックへの明示的な「ターンを1つ進める」メソッド呼び出しで進む。`MonoBehaviour` 側は描画と入力受付だけを担当し、ターン進行のタイミングを制御するに留める。
 - 物理を使わない: 移動・衝突判定はグリッド座標上の整数計算で行い、`Rigidbody`・`Collider` の物理に依存しない。見た目の演出に物理を使うのは可だが、勝敗を決めるロジックには関与させない。
 - 乱数は注入: 後述の制約Aの通り、`UnityEngine.Random` の静的メンバを使わない。
-

3. アーキテクチャの必須要件(コア層分離)

アセンブリ定義(`.asmdef`)でコードを2層に分ける。これは Unity Test Framework が Edit Mode テストでコアを直接テストするための前提でもある。

層	アセンブリ例	含むもの	禁止
コア層	<code>Game.Core.asmdef</code>	盤面生成・ターン進行・戦闘・リプレイ。プレーンな C# クラス	<code>UnityEngine</code> 依存(特に <code>MonoBehaviour</code> ・ <code>Random</code> ・ <code>Time</code> ・物理)の使用
提示層	<code>Game.Presentation.asmdef</code>	<code>MonoBehaviour</code> 、描画、入力、コア層の呼び出し	コア層のロジックをここに書くこと

コア層のアセンブリは `UnityEngine` を参照しない構成にできる。これによりコア層に Unity 依存が混入した瞬間にコンパイルエラーになり、分離が機械的に強制される。これが Unity 版で最も強力な制約である。

Unity Test Framework のドキュメント上、Edit Mode テストアセンブリは `nunit.framework.dll` を参照しテスト対象アセンブリを参照する構成が要求される。コア層を独立 `.asmdef` にすることがテスト可能性の前提になる。

4. 機械的制約(Unity 固有の判定手段)

汎用版の制約A~FをUnityの判定装置に差し替える。満たさない出力は動いても突き返す点は同じ。

制約A: Unity グローバル乱数の禁止

- 禁止: `UnityEngine.Random.Range` / `UnityEngine.Random.value` 等の静的メンバ。
- 必須: シードから生成した乱数源を引数注入する。選択肢は二つ。
 - `System.Random` のインスタンスをシード付きで生成し注入。
 - `Unity.Mathematics.Random` (構造体でシードを保持し、決定論的に進む) を注入。Unity 流ならこちら。
- 判定手段:
 - 第一選択: Roslyn Analyzer (C# コンパイラ拡張) で `UnityEngine.Random` の使用を検出しビルドエラーにする。
 - 簡易: CI で `grep -r "UnityEngine.Random\|Random.Range\|Random.value" Assets/` を走らせ、コア層にヒットしたら fail。
 - 補助: コア層 `.asmdef` を `UnityEngine` 非参照にしておけば、そもそも `UnityEngine.Random` がコンパイルできない (最も確実)。
- なぜ縛れるか: Claude は Unity で乱数が要ると反射的に `Random.Range` を書く。コア層を `UnityEngine` 非参照にしておくと、この反射がコンパイル段階で物理的に潰れる。

制約B: 決定論テストの常時グリーン

- 同一シードでコアロジックを2回走らせ、リプレイ列 (各ターンの状態列) が完全一致することを Edit Mode テストで固定。
- なぜ縛れるか: Claude が `Time`・物理・`UnityEngine.Random`・要素順序の不定なコレクションに依存すると即座に赤くなる。落ちたテストが客観的な突き返しの根拠になる。

制約C:マジックナンバー禁止

- 盤面サイズ・攻撃力・命中確率等をロジック中に直書きしない。`ScriptableObject` か定数クラスに集約。
- 判定手段: Roslyn Analyzer (マジックナンバー検出ルール)、または `.editorconfig` + Roslyn アナライザ (`dotnet` の CA 系・IDE 系ルール) を CI の `dotnet format --verify-no-changes` 相当で検査。
- 補足: Unity の数値はインスペクタ調整値が多いので、`ScriptableObject` への集約は Unity の作法とも一致する。

制約D: 例外・エラーの握り潰し禁止

- 空 `catch`、`catch (Exception) {}` の握り潰しを禁止。
- 判定手段: Roslyn Analyzer(CA1031 一般例外捕捉などのルール)を warning ではなく error 扱いに設定。

制約E: カバレッジ閾値

- コア層アセンブリの行カバレッジ 90% 未満で CI fail(計測手段は6章)。
- なぜ縛れるか: テストされていないコア層コードを出せなくする。

制約F: コア層の Unity 依存禁止(Unity 版では必須)

- 3章の通り、コア層 `.asmdef` を `UnityEngine` 非参照にする。`MonoBehaviour`・`Time`・`物理`・`Random` がコア層に入った瞬間コンパイルエラー。
- なぜ縛れるか: 汎用版では `import` リンターで実現していた境界を、Unity ではアセンブリ参照グラフそのもので強制できる。リンター以上に固い。

5. テスト基盤(Unity Test Framework / Edit Mode)

- **Edit Mode** テストを使う。Edit Mode テストはエディタ内でゲームループを回さず C# を直接実行できるため、決定論テストに適する。Edit モードテストはエディタ内でのみ実行され、エディタコードとランタイムのアプリケーションコードの両方にアクセスできる。
- **Play Mode** テストは決定論テストに使わない。物理やフレーム経過が絡むと再現性が崩れるため。描画や入力の統合確認に限定する。
- テストアセンブリは `nunit.framework.dll` を参照し、コア層アセンブリを参照する構成にする。通常の同期ロジックのテストは `UnityTest` 属性ではなく NUnit の `Test` 属性を使う(フレーム待ちが不要なため)。
- テストを先に書き、最初は赤、という TDD の順序は汎用版と同じ。

6. カバレッジ(Unity Code Coverage / バッチモード)

CI で Code Coverage パッケージをバッチモード計測する。Edit Mode テストを対象に、コア層アセンブリだけをフィルタする。

公式ドキュメントが示すバッチモードの計測コマンドは概ね次の形をとる(パスやアセンブリ名は自分の構成に合わせる)。`-batchmode -testPlatform editmode -runTests -enableCodeCoverage -coverageResultsPath <出力先> -coverageOptions` の形で

EditMode テストの計測を行い、別の起動で `generateHtmlReport;generateBadgeReport` を渡してレポートとバッジを生成する。

None

```
Unity -projectPath <project> -batchmode -testPlatform editmode -runTests \
  -debugCodeOptimization -enableCodeCoverage \
  -coverageResultsPath <results> \
  -coverageOptions "generateAdditionalMetrics;assemblyFilters:+Game.Core"
```

`assemblyFilters` で対象アセンブリを名前で絞り込める(例: `+my.assembly.*`)ため、コア層 `Game.Core` だけを計測対象にし、提示層やテストアセンブリを除外する。生成された結果(XML / バッジ)の数値を CI スクリプトで読み取り、90% 未満なら exit code を非ゼロにして fail させる。

閾値の自動判定そのものはパッケージの組み込み機能ではなく、生成された結果を CI スクリプト側で解釈して fail させる作りにするのが基本。`-debugCodeOptimization` を付けないと計測が正しく行われない点に注意。

7. 進め方・完成判定(汎用版からの差分)

進め方の原則は汎用版と同じ。制約の番人(`.asmdef` 構成・Analyzer 設定・CI・カバレッジ判定スクリプト)はユーザーが握り、Claude には「その機械判定を通すコードを書け」とだけ要求する。仕様書を丸投げして自発遵守に期待しない。

Unity 版で特に重要な初手は、コア層 `.asmdef` を `UnityEngine` 非参照で先に作っておくこと。これさえ用意すれば、Claude がコア層に `MonoBehaviour` や `Random.Range` を書いた瞬間にコンパイルが通らず、最も強い矯正が自動で効く。

完成判定(すべて同時に満たす):

- ☐ 決定論テスト(同一シードでリプレイ一致)が Edit Mode でグリーン。
 - ☐ ゴールデンテスト(既知シードで勝敗・ターン数固定)がグリーン。
 - ☐ コア層アセンブリのカバレッジ 90% 以上。
 - ☐ 制約A・C・D の Analyzer / grep 違反ゼロ。
 - ☐ コア層 `.asmdef` が `UnityEngine` を参照していない(制約F)。
 - ☐ 勝敗ロジックが物理・Time・UnityEngine.Random に依存していない。
-

8. 想定される Claude の逃げと対処 (Unity 版)

Claude の逃げ	兆候	対処 (突き返し方)
<code>UnityEngine.Random</code> をコア層で使う	コア層が <code>UnityEngine</code> 非参照ならコンパイルエラー／grep 違反	「乱数源を引数注入に直せ。コア層は <code>UnityEngine</code> 非依存」
ロジックを <code>MonoBehaviour</code> に書く	コア層に書けばコンパイルエラー、提示層に書けばカバレッジ計測外でテスト不能	「ロジックをコア層のプレーン C# に移せ」
<code>Time.deltaTime</code> でターン進行	決定論テストが赤	「ターン進行を明示的メソッド呼び出しにせよ。フレーム依存禁止」
<code>Rigidbody</code> ・物理で移動判定	決定論テストが赤	「移動はグリッド整数計算に。物理を勝敗ロジックから外せ」
要素順序が不定なコレクション依存	実行ごとにリプレイが微妙に変わる	「順序保証のあるデータ構造かソートを使え」
カバレッジ稼ぎの空テスト	カバレッジは満たすが assert がない	テスト本文を読み「実行しているだけで検証していない」を指摘
一般例外の握り潰し	制約 D の Analyzer 違反	「例外を表に出せ。握り潰し禁止」

各行が「Claude を矯正する」具体的場面である。Unity 版では、コア層 `.asmdef` の `UnityEngine` 非参照という1つの構成が、上位3～4行の逃げをコンパイル段階でまとめて封じる点が必要。